

Python Fundamentals

Final Project

Bringing It All Together

Throughout Python Fundamentals, you have built a toolbox for solving programming problems with Python.

During the course, we have worked with:

- variables, data types and operators
- strings and type conversion
- lists, tuples, sets and dictionaries
- conditions and loops
- functions, parameters and return values
- default and keyword arguments
- type hints and docstrings
- scope
- *args and **kwargs
- unpacking
- list, dictionary and set comprehensions
- enumerate() and zip()
- lambda functions
- map() and filter()
- classes and objects
- attributes, methods and object state
- instance and class attributes
- collections of objects
- objects containing other objects
- inheritance and super()
- method overriding
- polymorphism
- duck typing
- isinstance()
- __str__()
- IS-A and HAS-A relationships
- composition
- Validation

This Is Not a Checklist

You are not expected to use everything above in your project.

Think of what you have learned as a toolbox. A programmer does not use every tool in every program. An important part of programming is deciding which tool makes sense for the problem you are trying to solve.

Sometimes a list is enough. Sometimes a dictionary is a better choice. Sometimes there is a good reason to create a class. In some situations inheritance is a natural solution, while composition may be more appropriate in others.

Using more Python concepts does not automatically make a program better.

In this project, you will use what you have learned to **plan, build and develop a complete Python program**.

You will choose between three different project themes. All three have the same core expectations, but you have freedom in how you design and develop your solution.

The goal is to build a program that works, has a thoughtful structure and contains code and design choices that you understand and can explain.

Choose Your Project

Choose one of the following three project themes.

The descriptions and ideas below provide direction and inspiration. They do not define exactly how your program should be designed or which classes you should create. That part is up to you.

Option 1 - Zoo Simulation

Build a Python program that simulates life in a zoo.

Your zoo should contain different types of objects that interact with each other. Animals may have different characteristics and behaviours, visitors may interact with the zoo, and actions in the simulation should affect what happens later.

For example, animals could eat, rest, play or interact with visitors. Their condition or behaviour could change as the simulation progresses.

Your zoo could run through several activities, events or periods of time.

You decide what exists in your zoo, how the different parts are represented and how they interact.

The goal is not to create the most realistic zoo possible. The goal is to design a working system where objects, behaviour and state work together.

Possible ideas

- different animal behaviours
- feeding or care systems
- visitors and visitor activities
- animal health, hunger, energy or mood
- enclosures or different areas of the zoo
- scheduled activities
- statistics about the zoo
- events that affect the simulation
- several simulation rounds or days

Option 2 - Fantasy Adventure / RPG

Build a Python program that represents a fantasy adventure or RPG-style world.

Your program should contain different types of objects that can interact with each other.

Characters might explore, fight, heal, collect items, use abilities or complete different activities.

Actions should have consequences. A character might lose health during combat, an item might change what a character can do, or completing an activity might change the state of the game.

You decide what exists in your world, how characters and other objects are represented and how the different parts of the system interact.

The goal is not to build a complete video game. No graphics or graphical interface is required. Focus on the Python system and the logic behind the adventure.

Possible ideas

- different character types
- enemies
- health or other character statistics
- weapons, armor or other items
- inventory
- abilities
- combat
- quests or missions
- rewards
- locations
- character progression
- statistics about the adventure

Option 3 - Booking System

Build a Python program for managing bookings.

You decide what kind of booking system you want to create. It could be used for hotel rooms, sports facilities, meeting rooms, activities, appointments or another type of resource or service.

The system should contain different types of objects that interact with each other.

For example, a booking could connect a customer with something that can be booked.

Bookings should have meaningful behaviour. They might be created, cancelled, changed or checked in different ways.

The system should keep track of its current state while the program is running.

You decide how bookings, customers, resources and other parts of your system should be represented and how they relate to each other.

No database or permanent data storage is required. The data only needs to exist while the program is running.

Possible ideas

- different types of bookable resources or services
- customers
- availability
- booking status
- cancellation
- preventing conflicting bookings
- prices or costs
- different booking rules
- searching and filtering bookings
- statistics
- summaries of available and booked resources

Core Expectations

Regardless of which project you choose, the following describes the expected foundation of your project.

These expectations describe what your project should demonstrate, not exactly how you should build it.

1. Build a Complete and Connected Program

Your project should be one complete system rather than a collection of separate exercises or unrelated code examples.

The different parts of your program should work together.

2. Use Object-Oriented Programming

Your program should contain multiple classes with clear responsibilities.

Create objects from your classes and use them as meaningful parts of your system. Your classes should contain relevant attributes and methods.

3. Create Relationships and Interactions Between Objects

Objects in your system should not exist completely independently from each other.

They should interact, contain other objects, work with other objects or otherwise be connected as part of the system.

You decide how these relationships should be designed.

4. Actions Should Affect the State of Your System

Actions should have actual consequences in your program.

Something in your system should change as the program runs, and those changes should affect what can happen later.

The program should not simply create objects and print different messages.

5. Work With Collections of Data or Objects

Your program should manage multiple pieces of information or multiple objects using appropriate Python collections.

Choose the collection types that make sense for your solution.

6. Include Meaningful Program Logic

Use conditions, loops, functions and/or methods to solve problems and control how the system behaves.

Avoid building a program that mainly consists of hard-coded values and print statements.

7. Handle Invalid or Unreasonable Actions

Your program should prevent or handle situations that do not make sense within your system.

What those situations are will depend on your project. You are responsible for deciding which situations your program needs to protect against.

8. Organize the Project Into Multiple Python Modules

Your complete project should not be implemented in one large Python file.

Divide your code into multiple .py files in a way that makes sense for your program.

There is no required number of files or required file structure. The important part is that your organization has a purpose.

9. Make the Program Runnable and Demonstrable

There should be a clear way to start your project and demonstrate its main functionality.

The program should run without errors during its normal intended use.

10. Use Git Throughout the Project

Commit and push regularly as your solution develops.

Prefer small, meaningful commits that represent a clear change, improvement or completed piece of functionality.

Avoid saving most of the project in one or two large commits at the end.

Your Git history should give a reasonable picture of how your project developed.

Examples:

```
Add basic booking functionality
Prevent conflicting bookings
Add visitor interactions
Implement character health system
Refactor shared behaviour
Fix negative energy bug
```

Further Development

Once your core system works, think about how you can improve its design, behaviour and readability.

Depending on your project, you may consider:

- reducing duplicated code
- separating responsibilities between classes and functions
- improving how objects interact
- using composition when a HAS-A relationship makes sense
- using inheritance when there is a meaningful IS-A relationship
- using method overriding and polymorphism where appropriate
- using `__str__()` when objects need a useful human-readable representation
- using comprehensions, `enumerate()`, `zip()`, unpacking, lambda functions or other Python tools when they make the solution clearer
- adding useful validation
- considering unusual situations and edge cases
- adding filtering, sorting, searching or statistics where appropriate
- improving names, structure and readability
- extending the system with additional meaningful functionality

These are suggestions for developing your project further. Choose techniques and improvements that make sense for your solution rather than trying to include everything.

Suggested Project Workflow

This is a suggested workflow, not a required schedule.

You may organize your work differently depending on your project and previous programming experience. You may also move back and forth between the phases as your solution develops.

Phase 1 - Plan and Build the Foundation

Start by deciding what your system actually needs to do.

Think about:

- What are the most important parts of the system?
- What objects might exist?
- What responsibilities should they have?
- How should different parts of the system interact?
- What is the smallest version of the project that would actually work?

Then start implementing the foundation.

Do not try to design the entire project perfectly before you start coding. Your design will probably change as you build.

Phase 2 - Make the Core System Work

Focus on the most important functionality.

Try to get a simple version of the complete system working from beginning to end before adding lots of additional features.

Zoo: animals and other parts of the zoo can interact and state can change.

RPG: characters can perform meaningful actions that affect the game state.

Booking: bookings can be created and managed and the system keeps track of its current state.

A simple working system is more valuable than ten unfinished features.

Phase 3 - Review Your Design

Once you have a working foundation, look at your code again.

Ask yourself:

- Are responsibilities divided clearly?
- Are any classes doing too much?
- Is code duplicated?
- Are the modules organized logically?
- Are objects interacting in a sensible way?
- Would composition improve part of the design?
- Is there a genuine IS-A relationship where inheritance could help?
- Could polymorphism simplify some behaviour?

Refactor where it improves the program.

Do not add inheritance, additional classes or other techniques simply because they exist in Python.

Phase 4 - Improve and Test

Test your program in different situations, not only the happy path where everything works exactly as expected.

Think about situations such as:

- invalid values
- actions performed in the wrong state
- empty collections
- repeated actions
- unavailable resources
- unusual combinations of actions

Which situations matter will depend on your project.

Fix problems you discover and improve the parts of the program that need it.

Once the core system is stable, you can also develop additional functionality that genuinely improves your project.

Phase 5 - Finish, Clean and Demonstrate

Before considering your project finished:

- finish incomplete functionality
- remove unnecessary or unused code
- improve variable, function and class names
- check your project structure
- test the complete program again
- make sure the project starts correctly
- make sure the main functionality can be demonstrated
- make sure your latest work has been pushed to GitHub

Finally, ask yourself:

Can I explain how my program works and why I designed it this way?

You should understand your code and be able to explain the important decisions behind your solution.

When You Are Finished

Make sure the latest version of your project is pushed to GitHub and that the repository contains everything needed to run the program.

Include a short README.md explaining:

- what your project does
- its main functionality
- how to run the program

The README should be short and clear. It is not a separate report.